

# Real-Time Model Monitoring & Alerting Stack

MLOps Project 4 — Project Report

Team 4 · wrapped around the Telco Customer-Churn model

Repository: <https://github.com/AutomationArtist01/MLOPS>

Full technical guidebook (why every tool, alternatives, PromQL, demo, Q&A): `docs/Team4-Monitoring-Guidebook.pdf`

# 1 · Objective

Machine-learning services fail *silently*: an endpoint keeps answering 200 OK while the model's input distribution drifts away from the training data, or a serving bug scrambles the features. Infrastructure monitoring (CPU, memory, uptime) cannot see this.

**Objective.** Build a **reusable, model-agnostic observability stack** that gives any deployed model — starting with the Telco Customer-Churn classifier built by the *customer-churn-mlops* team — real-time visibility of its **operational health** (request rate, latency p50/p95/p99, error rate) and its **ML health** (prediction distribution, data drift via PSI and the KS-test on a rolling window), and that **alerts humans** through Slack and e-mail via Alertmanager, all orchestrated with Docker Compose and documented so it can be wired to any new model service.

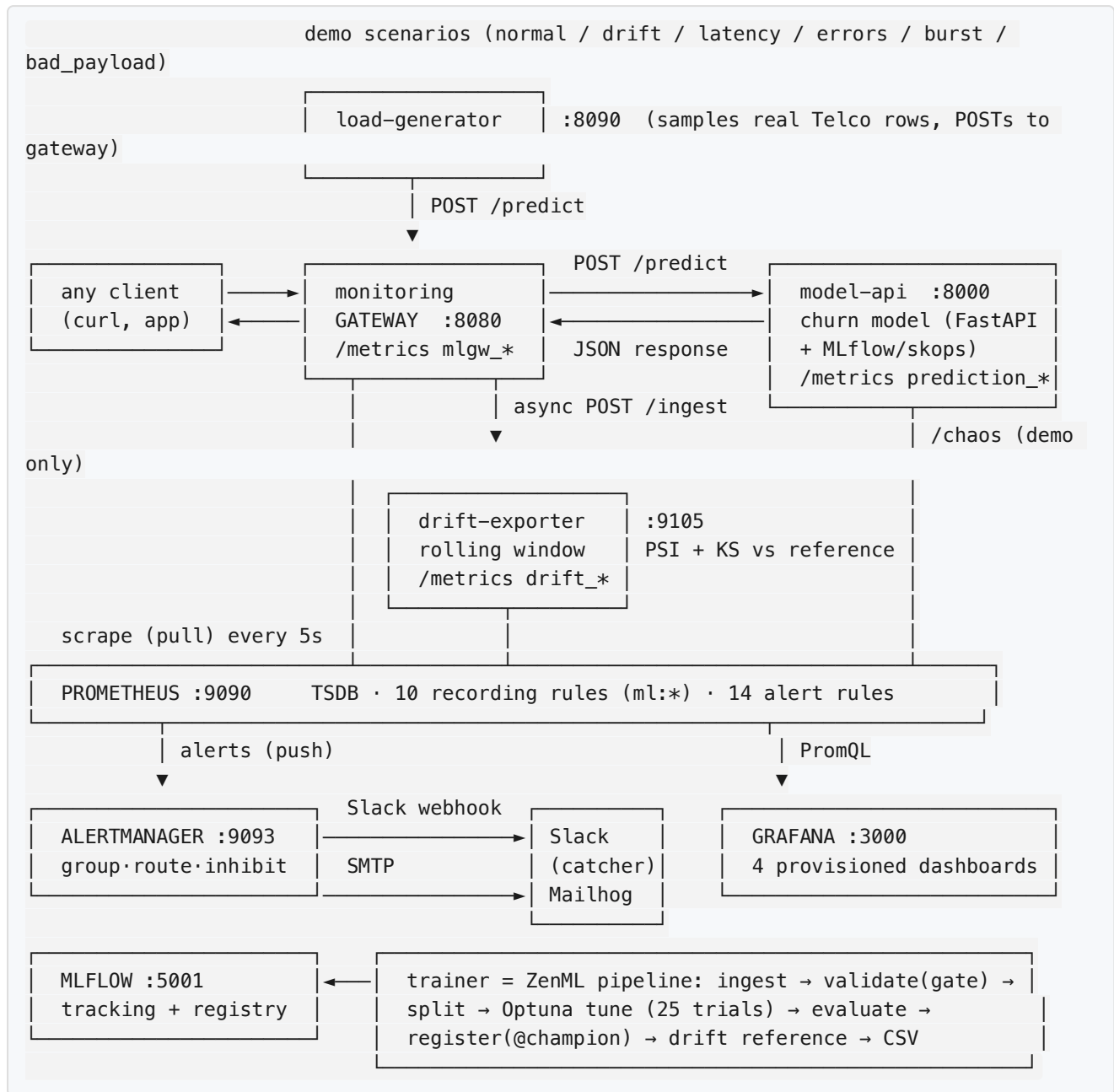
**Dataset.** IBM Telco Customer Churn — 7 043 customers, 19 features (4 numeric, 15 categorical), 26.5 % churners. It is the dataset the wrapped model was trained on, so it is used (a) to retrain and tune the model in our own ZenML + Optuna pipeline, (b) as the drift *reference* distribution and (c) to replay realistic traffic. No new dataset was required by the assignment.

**Success metrics (defined up-front).**

| Area            | Metric                                        | Target                                  | Achieved                                             |
|-----------------|-----------------------------------------------|-----------------------------------------|------------------------------------------------------|
| Model quality   | hold-out ROC-AUC (primary), F1                | $\geq 0.84$ · registry gate $\geq 0.80$ | <b>0.847 AUC / 0.587 F1</b> (CV AUC 0.850)<br>✓      |
| Serving SLOs    | p95 latency · 5xx ratio · availability (30 m) | $< 500$ ms · $< 5$ % · $\geq 99$ %      | p95 $\approx 25$ ms baseline; alerts fire on breach  |
| Drift detection | PSI / KS per feature; time-to-detect          | PSI $> 0.25$ flagged $\leq 3$ min       | 6 features flagged $\sim 2$ min after shift          |
| Alerting        | routing correctness, grouping                 | 100 % of scenarios delivered            | 12 Slack + 4 e-mail notifications, correct channels  |
| Reusability     | wiring a new model                            | env vars + 1 scrape job                 | proven with the other team's <i>unmodified</i> image |

**Hour budget (15 h).** Exporters 5 h · Docker Compose 3 h · Dashboards 3 h · Alertmanager 2 h · Docs/demo 2 h.

## 2 · Architecture



Every box is a container in `docker-compose.yml`. Prometheus also scrapes itself, Alertmanager, Grafana and MLflow (meta-monitoring). The **gateway** makes the stack model-agnostic: it proxies `/predict` to any HTTP model (`UPSTREAM_URL`, `PREDICT_PATH`, `PROB_FIELD`, `LABEL_FIELD` env vars), records the four operational signals + prediction distribution with a `model` label, and streams feature rows to the drift exporter asynchronously (never on the request path).

**Key design decisions** (details and alternatives in the guidebook): Prometheus *pull* model (free health detection, central config) · **histograms** not summaries (server-side p50/p95/p99, aggregatable) · gateway/sidecar instrumentation **plus** in-process metrics in the model (both shown, trade-offs documented) · PSI (numeric + categorical, interpretable thresholds 0.10/0.25) **and** KS-test (bin-free statistical test) on a 500-row count-based rolling window · Prometheus alert rules → Alertmanager (versioned YAML, grouping, inhibition, silences) rather than Grafana-managed alerts · dashboards generated as code · Docker Compose over Kubernetes for a single-host, one-command demo.

## 3 · Implementation

### 3.1 Data pipeline & orchestration (ZenML)

`services/trainer/pipeline.py` — `@pipeline churn_training_pipeline` with eight `@steps`:

| Step                               | What it does                                                                                                                                                                 | Gate                                  |
|------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------|
| <code>ingest_data</code>           | reads the CSV                                                                                                                                                                | —                                     |
| <code>validate_data</code>         | schema (23 required columns), <code>TotalCharges</code> coercion, null share $\leq 5\%$ , target $\in \{\text{Yes, No}\}$ , minority class $\geq 10\%$ , non-negative tenure | <b>raises</b> → <b>pipeline halts</b> |
| <code>split_data</code>            | stratified 80/20, target → 0/1                                                                                                                                               | —                                     |
| <code>tune_and_train</code>        | Optuna TPE + MedianPruner, 25 trials, 3-fold CV ROC-AUC, <b>every trial a nested MLflow run</b> ; best config retrained on full train split                                  | —                                     |
| <code>evaluate_model</code>        | hold-out AUC/accuracy/precision/recall/F1 logged to the parent run with the model artifact (signature + input example)                                                       | —                                     |
| <code>register_model</code>        | promotes to <code>telco-churn-classifier</code> with alias <code>champion</code> and tag <code>stage=production</code>                                                       | <b>refuses AUC &lt; 0.80</b>          |
| <code>build_drift_reference</code> | writes the PSI/KS reference profile (quantile bins, category shares, training-score histogram) for the drift exporter                                                        | —                                     |
| <code>export_runs_csv</code>       | all runs → <code>artifacts/mlflow_runs.csv</code>                                                                                                                            | —                                     |

Steps are cached by ZenML (except tuning) and artifacts are versioned in ZenML's store; the unit tests exercise the validation gate on real data with a missing column and an artificially imbalanced target (both halt).

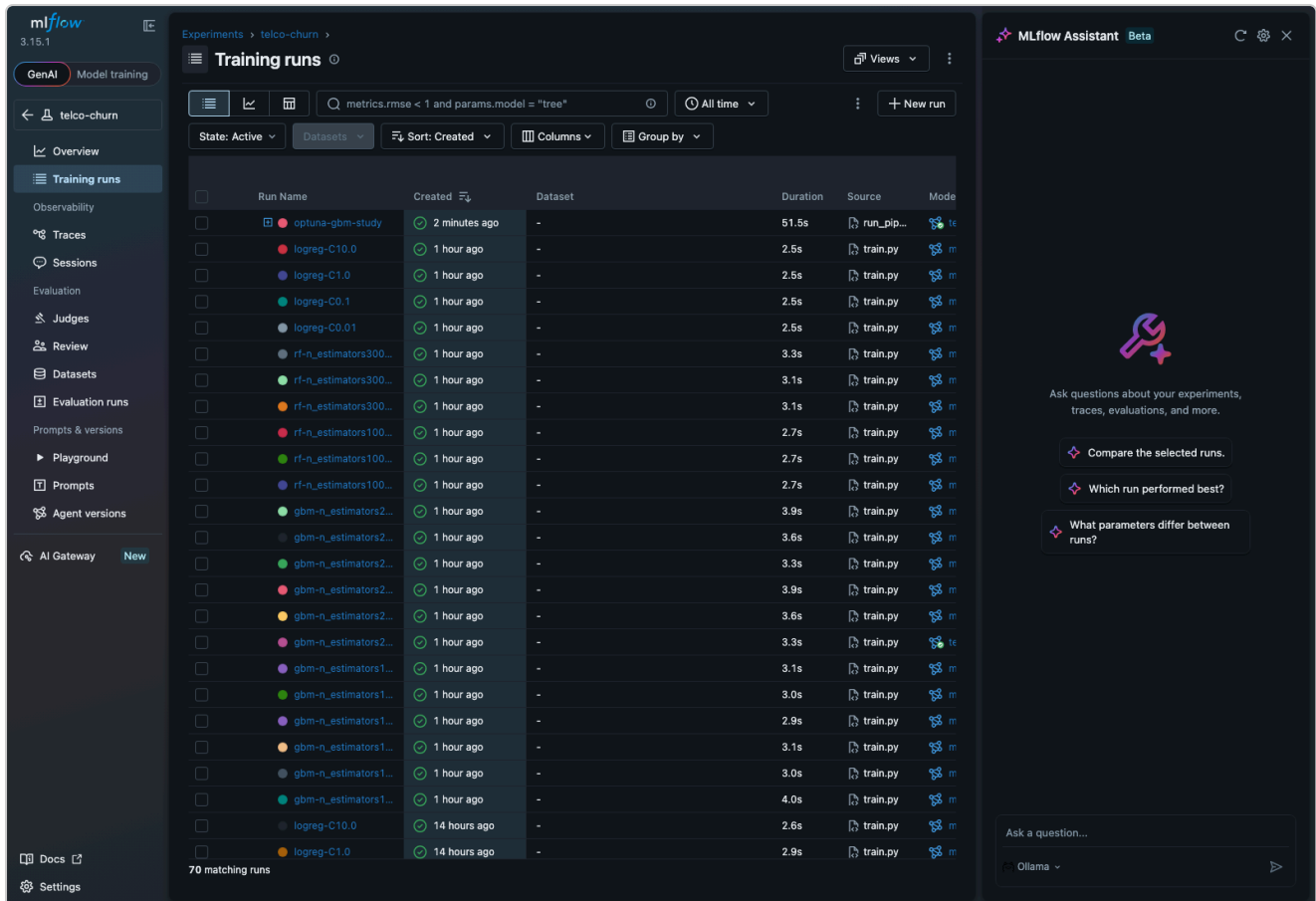
```
$ docker compose run --rm trainer -- python run_pipeline.py (ZenML · churn_training_pipeline · 25 Optuna trials)

Initiating a new run for the pipeline: churn_training_pipeline.
Registered new pipeline: churn_training_pipeline.
Step ingest_data has started.
[ingest] /data/telco_churn.csv: 7043 rows × 21 cols
Step ingest_data has finished in 0.317s.
Step validate_data has started.
[validate] {"rows": 7043, "null_share": {"TotalCharges": 0.0016}, "churn_rate": 0.2653698707936959, "problems": {}}
Step validate_data has finished in 0.211s.
Step split_data has started.
[split] train 5634 / test 1409 · churn rate 0.265
Step split_data has finished in 0.814s.
Step tune_and_train has started.
[I 2026-08-18 06:36:51.491] Trial 0 finished with value: 0.8044262139885077 and parameters: {'n_estimators': 150, 'learning_rate': 0.2536999076681772, 'max_depth': 5, 'min_samples_split': 13, 'min
[I 2026-08-18 06:36:52.495] Trial 1 finished with value: 0.8391996163136096 and parameters: {'n_estimators': 50, 'learning_rate': 0.19030368381735815, 'max_depth': 5, 'min_samples_split': 15, 'min
[I 2026-08-18 06:36:54.316] Trial 2 finished with value: 0.8494733209940802 and parameters: {'n_estimators': 275, 'learning_rate': 0.020589728197687916, 'max_depth': 2, 'min_samples_split': 5, 'min
[I 2026-08-18 06:36:56.335] Trial 3 finished with value: 0.8477233826222745 and parameters: {'n_estimators': 150, 'learning_rate': 0.02692655251486473, 'max_depth': 5, 'min_samples_split': 4, 'min
[I 2026-08-18 06:36:57.399] Trial 4 finished with value: 0.845032173641977 and parameters: {'n_estimators': 175, 'learning_rate': 0.14447746112718687, 'max_depth': 2, 'min_samples_split': 11, 'min
[I 2026-08-18 06:36:58.910] Trial 5 finished with value: 0.8464535649052619 and parameters: {'n_estimators': 200, 'learning_rate': 0.0178601378893971, 'max_depth': 2, 'min_samples_split': 20, 'min
[I 2026-08-18 06:37:00.732] Trial 6 finished with value: 0.848042823043914 and parameters: {'n_estimators': 125, 'learning_rate': 0.013940346079873234, 'max_depth': 5, 'min_samples_split': 10, 'min
[I 2026-08-18 06:37:01.240] Trial 7 pruned.
[I 2026-08-18 06:37:03.779] Trial 8 pruned.
[I 2026-08-18 06:37:04.721] Trial 9 pruned.
[I 2026-08-18 06:37:06.678] Trial 10 pruned.
[I 2026-08-18 06:37:10.081] Trial 11 finished with value: 0.8491799620920876 and parameters: {'n_estimators': 300, 'learning_rate': 0.010446609340209814, 'max_depth': 4, 'min_samples_split': 8, 'min
[I 2026-08-18 06:37:12.981] Trial 12 finished with value: 0.8492350343068101 and parameters: {'n_estimators': 300, 'learning_rate': 0.01101278928574014, 'max_depth': 3, 'min_samples_split': 7, 'min
[I 2026-08-18 06:37:15.307] Trial 13 finished with value: 0.8484675964297291 and parameters: {'n_estimators': 250, 'learning_rate': 0.037370046395123875, 'max_depth': 3, 'min_samples_split': 7, 'min
[I 2026-08-18 06:37:16.997] Trial 14 pruned.
[I 2026-08-18 06:37:19.664] Trial 15 finished with value: 0.8494848009036694 and parameters: {'n_estimators': 250, 'learning_rate': 0.010088777799749494, 'max_depth': 4, 'min_samples_split': 2, 'min
[I 2026-08-18 06:37:22.344] Trial 16 finished with value: 0.8483195080017291 and parameters: {'n_estimators': 250, 'learning_rate': 0.02604358152505174, 'max_depth': 4, 'min_samples_split': 2, 'min
[I 2026-08-18 06:37:27.045] Trial 18 pruned.
[I 2026-08-18 06:37:29.366] Trial 19 finished with value: 0.8492084233970817 and parameters: {'n_estimators': 225, 'learning_rate': 0.014525573478714747, 'max_depth': 4, 'min_samples_split': 10, 'm
[I 2026-08-18 06:37:30.905] Trial 20 finished with value: 0.8497844935388331 and parameters: {'n_estimators': 275, 'learning_rate': 0.03543965171315069, 'max_depth': 2, 'min_samples_split': 5, 'min
[I 2026-08-18 06:37:32.435] Trial 21 finished with value: 0.8495458022372618 and parameters: {'n_estimators': 275, 'learning_rate': 0.03892786471849319, 'max_depth': 2, 'min_samples_split': 5, 'min
[I 2026-08-18 06:37:33.533] Trial 22 pruned.
[I 2026-08-18 06:37:34.884] Trial 23 finished with value: 0.8497535440230962 and parameters: {'n_estimators': 225, 'learning_rate': 0.03704001852811789, 'max_depth': 2, 'min_samples_split': 8, 'min
[I 2026-08-18 06:37:36.237] Trial 24 finished with value: 0.8499492578162591 and parameters: {'n_estimators': 225, 'learning_rate': 0.03789002839130302, 'max_depth': 2, 'min_samples_split': 9, 'min
[optuna] 25 trials (8 pruned) · best cv AUC 0.8499 · {'n_estimators': 225, 'learning_rate': 0.03789002839130302, 'max_depth': 2, 'min_samples_split': 9, 'min_samples_leaf': 9, 'subsample': 0.645642
Step tune_and_train has finished in 47.723s.
Step evaluate_model has started.
[evaluate] {"test_roc_auc": 0.847, "test_accuracy": 0.8048, "test_precision": 0.6701, "test_recall": 0.5214, "test_f1": 0.5865}
Step evaluate_model has finished in 3.898s.
Step register_model has started.
Created version '3' of model 'telco-churn-classifier'.
[registry] telco-churn-classifier v3 → @champion (stage=production)
Step register_model has finished in 0.714s.
Step build_drift_reference has started.
[reference] wrote /artifacts/telco_reference.json
Step build_drift_reference has finished in 0.054s.
Step export_runs_csv has started.
[csv] 70 runs → /artifacts/mlflow_runs.csv
Step export_runs_csv has finished in 0.109s.
Pipeline run has finished in 54.078s.
[zenml] pipeline run: churn_training_pipeline-2026_08_18-06_36_47_919531 · status=completed
```

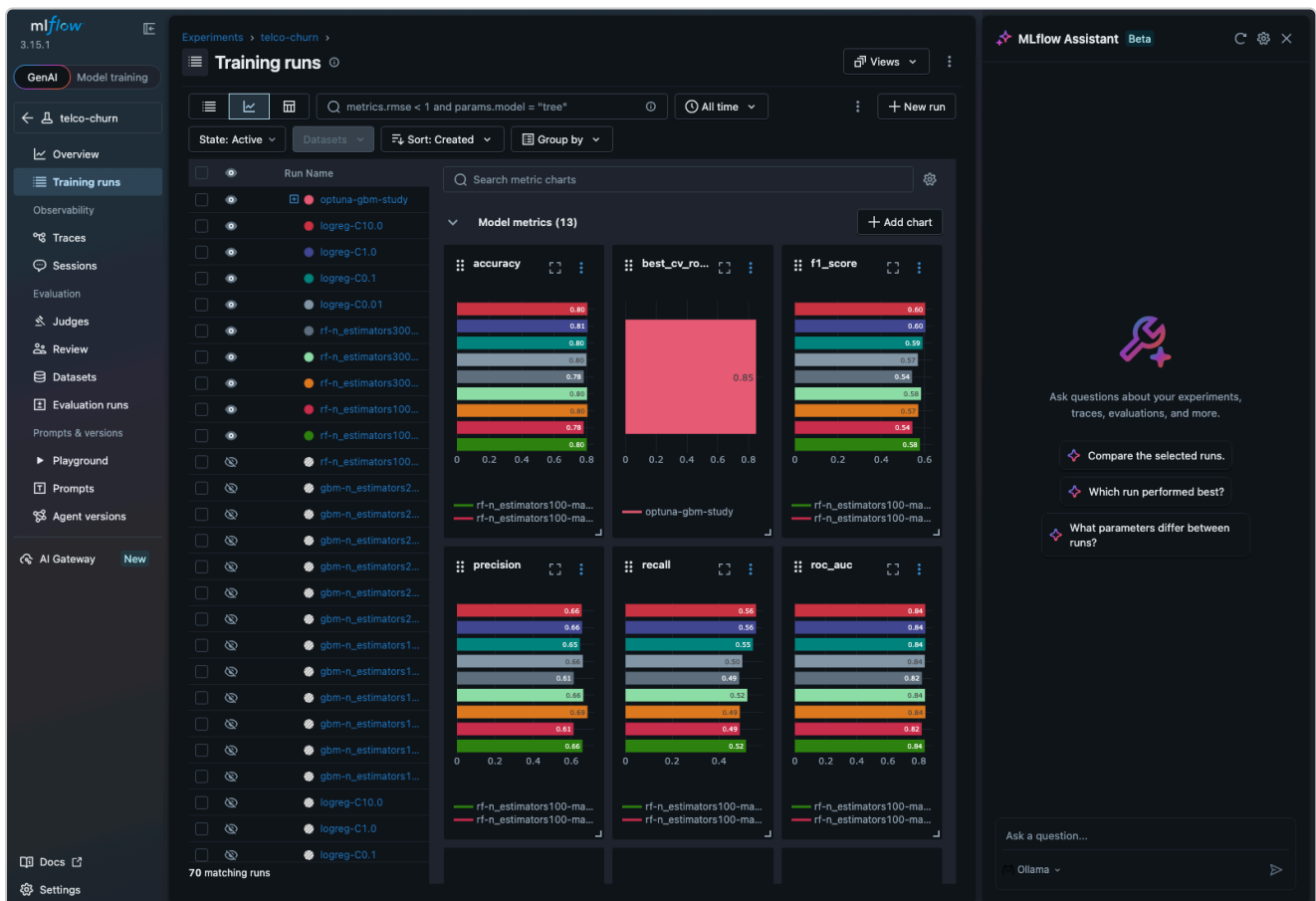
ZenML pipeline run log — steps, validation report, Optuna summary, registry promotion

## 3.2 Experiment tracking & tuning (MLflow + Optuna)

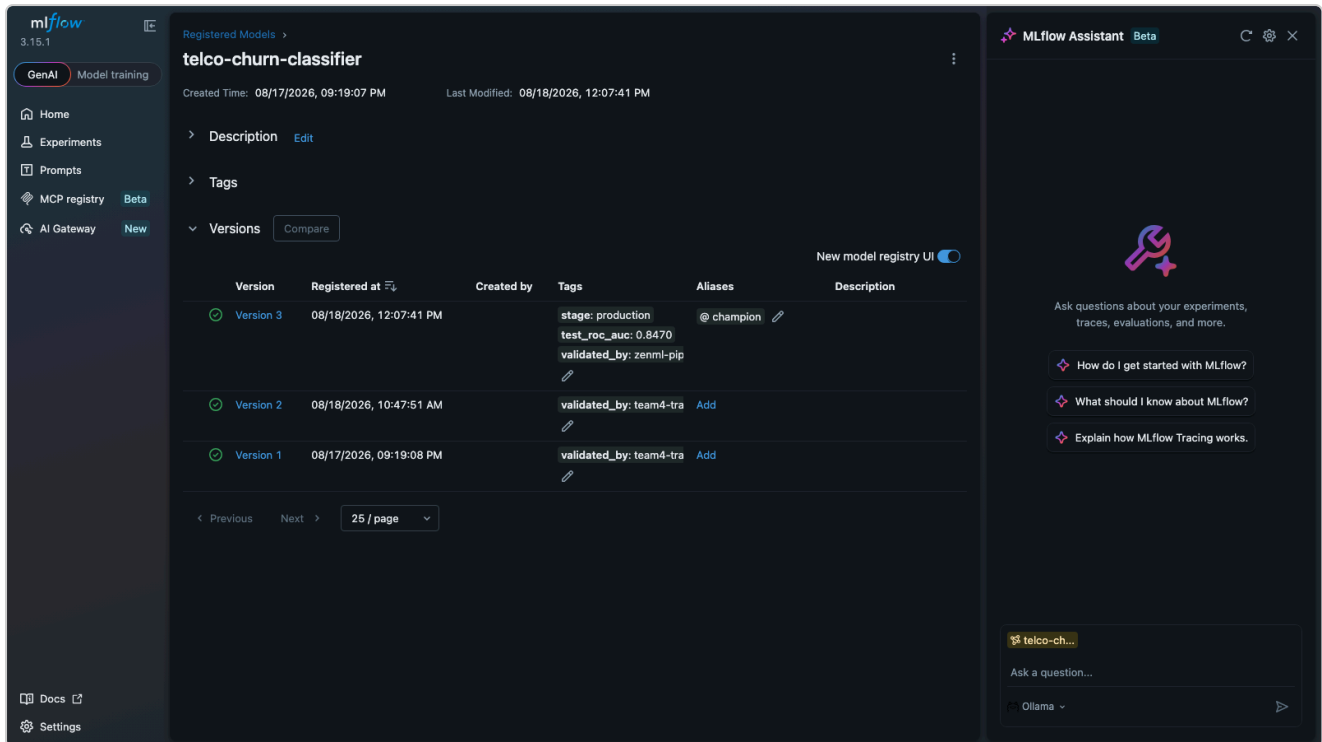
Search space: `n_estimators` 50–300, `learning_rate` 0.01–0.3 (log), `max_depth` 2–6, `min_samples_split` 2–20, `min_samples_leaf` 1–10, `subsample` 0.6–1.0 (GradientBoostingClassifier, same family the other team used). Sampler **TPE** (seeded), pruner **Median** (partial-CV reporting) — pruned trials are logged as such. Parent run holds `best_cv_roc_auc`, `trials_pruned`, the per-trial table (`optuna_trials.json`) and parameter importances (`optuna_param_importance.json`).



MLflow — nested Optuna trial runs, sortable/filterable by params & metrics



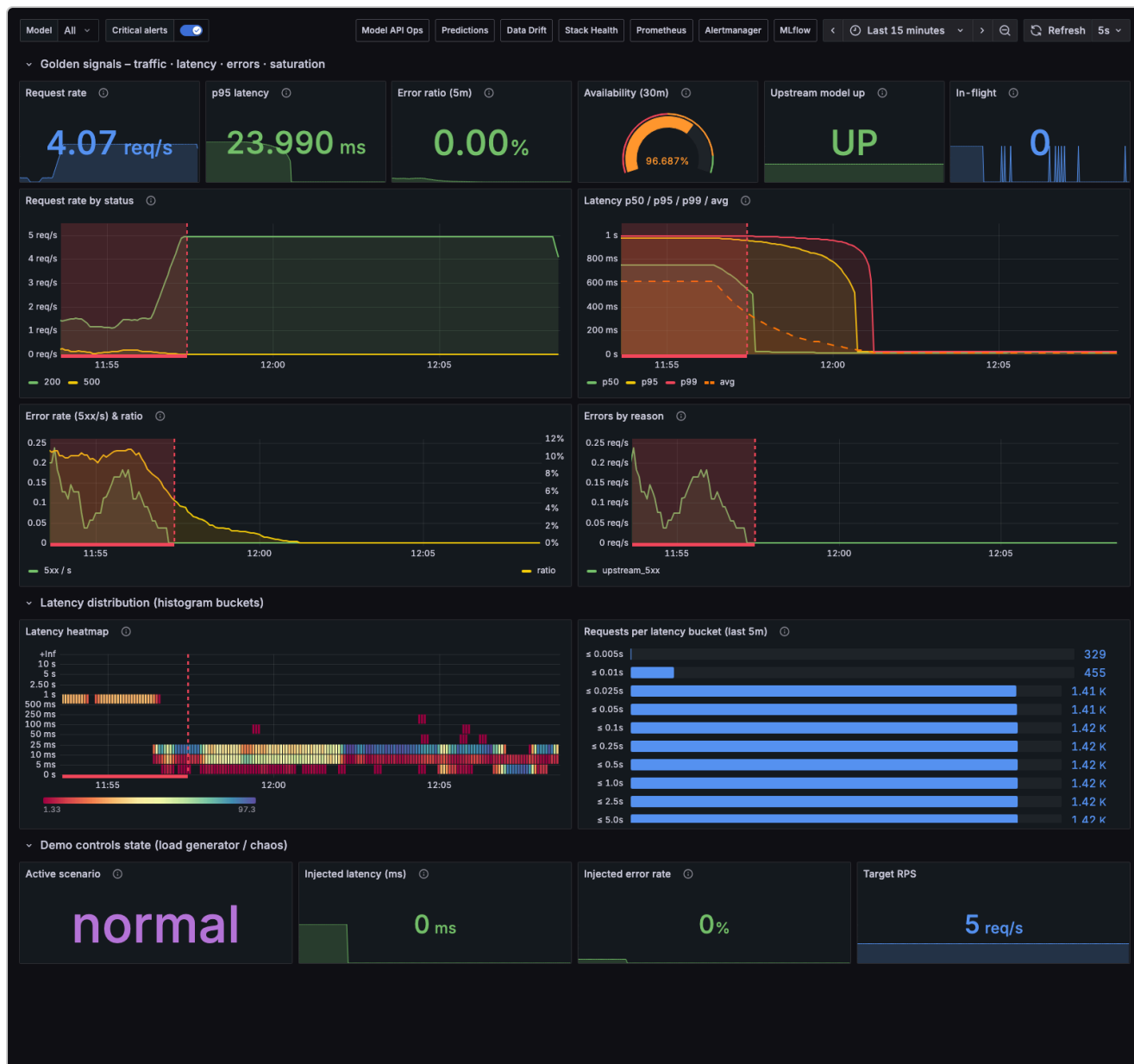
MLflow — parallel-coordinates / scatter of hyper-parameters vs CV AUC



MLflow Model Registry — telco-churn-classifier, alias @champion, stage tag

### 3.3 Monitoring & observability (Prometheus + Grafana + Alertmanager)

- **Exporters in service code:** `services/model_api/api.py` (`prediction_requests_total`, `prediction_request_latency_seconds` histogram, `prediction_errors_total{reason}`, `prediction_probability` histogram, `predictions_total{predicted_class}`, `model_info`) and the model-agnostic `services/gateway/gateway.py` (`mlgw_*` equivalents + `mlgw_upstream_up`, `mlgw_inflight_requests`).
- **Custom drift exporter** `services/drift_exporter/drift_exporter.py`: `drift_psi{feature,type}`, `drift_ks_statistic/pvalue{feature}`, `drift_prediction_psi`, `drift_features_drifted`, window and liveness gauges;  $PSI = \sum(a-e) \cdot \ln(a/e)$  on reference-quantile bins (open outer bins,  $\epsilon$ -smoothing, "other" bucket for unseen categories); two-sample KS from `scipy`.
- **Prometheus** `monitoring/prometheus/prometheus.yml` (9 jobs, 5 s scrape) + 10 recording rules (SLIs: `ml:latency_p95:5m`, `ml:error_ratio:5m`, `ml:availability:30m`, `ml:positive_rate:5m` ...)
- 14 alert rules in two groups (operational / ML-quality) with `for:` durations, severities and runbook links.
- **Alertmanager:** routes by severity/category to Slack channels and e-mail, grouping, three inhibition rules, custom Slack template; env-substituted so a real webhook/SMTP is one `.env` away.
- **Grafana:** four dashboards generated by `monitoring/grafana/build_dashboards.py` (57 panels, units, thresholds, `$model` variable, alert annotations), provisioned with the Prometheus and Alertmanager data sources.

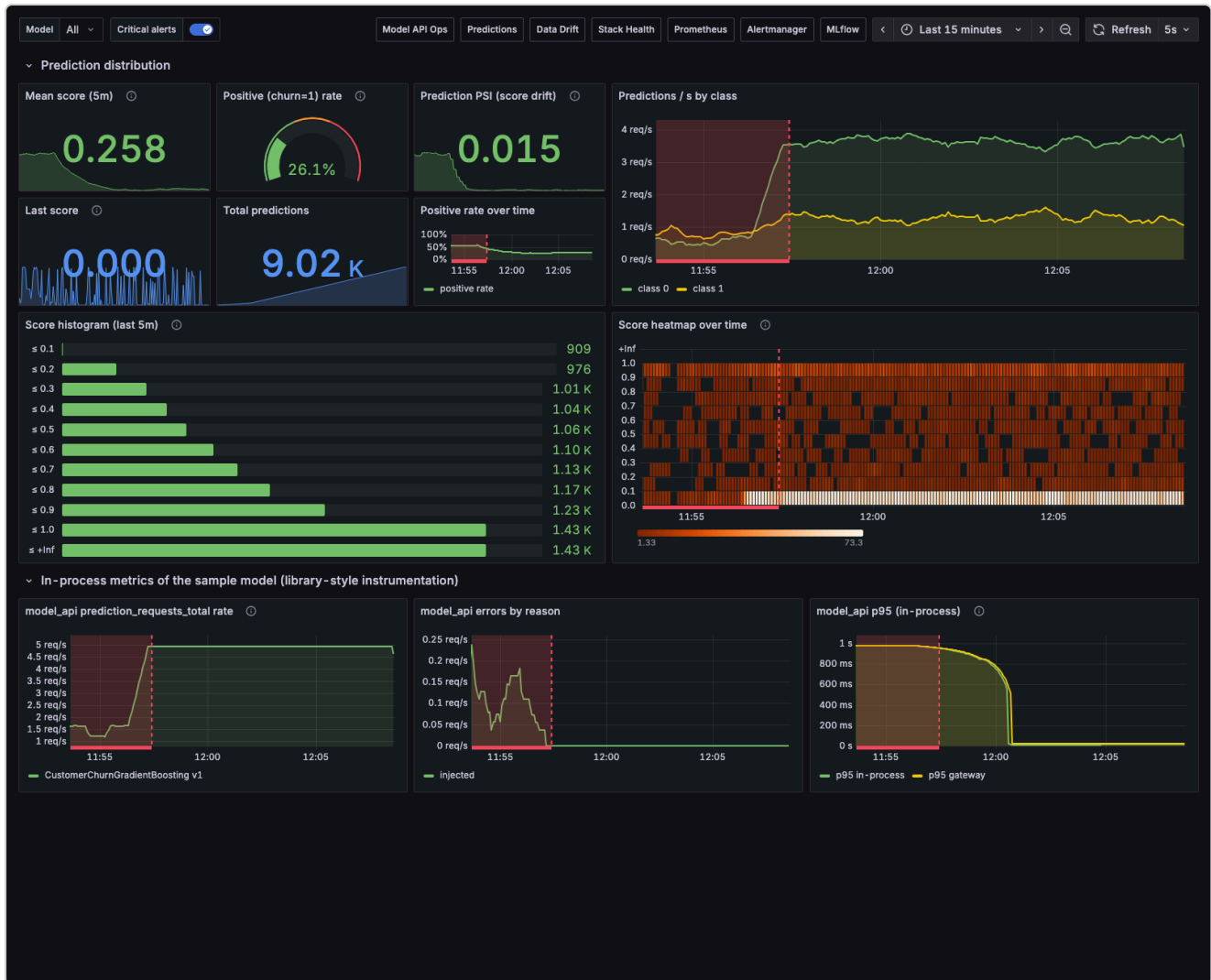


Grafana 1 — Model API golden signals during the latency scenario





Grafana 3 — Data drift: PSI per feature, PSI/KS over time, live vs reference means



Grafana 2 — Prediction distribution and positive rate

## 3.4 Load generator & demo scenarios

`services/load_generator` replays real customers through the gateway (5 rps) and exposes an API to switch scenarios (`make scenario=drift | latency | errors | burst | bad | normal`): drift shifts tenure/MonthlyCharges/Contract/InternetService/PaymentMethod; latency and errors are injected via a demo-only `/chaos` endpoint of the model API (`CHAOS_ENABLED=0` in production/Render).

## 3.5 Tests

`tests/test_units.py` (pytest, in CI): PSI/KS maths, drift detection end-to-end on the exporter object, gateway JSON-path helper, reference builder CLI, ZenML validation gate (pass / missing column / imbalance), Prometheus/alert-rule/dashboard config sanity. `tests/smoke_test.py` (23 assertions) boots against the running stack in CI and locally (`make smoke`).

## 4 • Deployment

---

- **Containerisation** — one `Dockerfile` per service ( `python:3.12-slim`, requirements layer cached before code copy, pinned dependencies via `uv pip compile`, `HEALTHCHECK` on the model API, `PORT`-aware start command). Official images for Prometheus, Alertmanager, Grafana, Mailhog.
- **Docker Compose** — 10 services, named volumes for Prometheus/Grafana/Alertmanager/MLflow/ZenML state, read-only bind mounts for configuration, `restart: unless-stopped`, `trainer` under the `train` profile, `docker-compose.second-model.yml` overlay to attach a second model.
- **Render** — `render.yaml` blueprint deploys `services/model_api` as a Docker web service with `/health` as health-check path (free plan). Live URL: **see repository README / LMS submission** — the local stack monitors it by pointing a gateway's `UPSTREAM_URL` at the Render URL.
- **CI/CD** — `.github/workflows/ci.yml` on every push/PR: `Ruff lint` → `pytest` → `promtool check config/rules` → `amtool check-config` → `docker compose config` → dashboards reproducible from code → **build all six images** (pushed to GHCR on `main`) → **compose smoke test** (boots the whole stack in the runner and runs the 23 assertions).

**Evidence pending at the time of writing:** the GitHub Actions run badge/screenshot and the live Render URL are added to this section (and to the README) after the repository is pushed and the Render blueprint is applied — both are one-click actions on the team's accounts. The workflow file ( `.github/workflows/ci.yml` ) and blueprint ( `render.yaml` ) are in the repository; the local equivalent of the CI job ( `make lint test smoke` ) passes: `ruff` clean, 11 unit tests, 23 smoke assertions.

## 5 • Results

### 5.1 Model (ZenML + Optuna run)

| Metric                                      | Value                                                                                                         |
|---------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| Optuna trials / pruned                      | 25 / 8 (MedianPruner)                                                                                         |
| Best CV ROC-AUC                             | 0.8499                                                                                                        |
| Best params                                 | n_estimators=225, learning_rate=0.0379, max_depth=2, min_samples_split=9, min_samples_leaf=9, subsample=0.646 |
| Hold-out ROC-AUC                            | 0.847                                                                                                         |
| Hold-out F1 / precision / recall / accuracy | 0.587 / 0.670 / 0.521 / 0.805                                                                                 |
| Registered                                  | telco-churn-classifier v3 → alias champion, tag stage=production                                              |

(Other team's baseline: Optuna 35 trials, ROC-AUC 0.845 on their split.)

### 5.2 Monitoring scenarios (all verified live)

| Scenario                                   | Observed                                                                    | Alerts fired                                                                                | Routed to                               |
|--------------------------------------------|-----------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|-----------------------------------------|
| normal (5 rps)                             | p95 ≈ 25 ms, error ratio 0 %, PSI < 0.15 on all features                    | —                                                                                           | —                                       |
| latency (700 ms injected)                  | p95 ≈ 0.9 s, p99 ≈ 1 s, avg ≈ 0.7 s                                         | HighLatencyP95                                                                              | Slack #ml-alerts                        |
| errors (30 % injected)                     | error ratio 16 % over 5 m                                                   | HighErrorRate (critical)                                                                    | Slack #ml-oncall + e-mail               |
| drift (population shift)                   | PSI up to 5.4 on tenure, 6 features > 0.25, KS D > 0.4, prediction PSI 0.81 | FeatureDriftHigh x6, FeatureDriftKS x3, MultipleFeaturesDrifted (critical), PredictionDrift | Slack #ml-quality / #ml-oncall + e-mail |
| second model (other team's original image) | monitored with zero code changes; same customer scores 0.069 vs 0.087       | —                                                                                           | —                                       |

AlertmanagerAlertsSilencesStatusSettingsHelp

New Silence

FilterGroup

Receiver: AllSilencedInhibitedMuted

+

Silence

Custom matcher, e.g. `env="production"`

Expand all groups

No alert groups found

Alertmanager — grouped alerts and receivers during the drift scenario

### Fake Slack – 19 notifications

```
06:31:04 /slack
{
  "channel": "#ml-alerts",
  "username": "Alertmanager",
  "attachments": [
    {
      "title": "[RESOLVED] HighLatencyP95 \u2013 warning",
      "title_link": "http://68617e369671:9093/#/alerts?receiver=team4-slack",
      "text": "\u2013p95 latency 514.7ms > 500ms for CustomerChurnGradientBoosting*\n95th percentile prediction latency over 5m is above the 500ms SLO.\nmodel: 'CustomerChurnGradientBoosting' feature: ''\nsince: 06:05:10\nrunbook: https://github.com/AutomationArtist01/MLOPS/blob/main/docs/Team4-Monitoring-Guidebook.pdf#highlatencyp95\n",
      "fallback": "[RESOLVED] HighLatencyP95 CustomerChurnGradientBoosting warning (operational ml-monitoring team4) | http://68617e369671:9093/#/alerts?receiver=team4-slack",
      "callback_id": "",
      "footer": "",
      "color": "good",
      "mrkdwn_in": [
        "fallback",
        "pretext",
        "text"
      ]
    }
  ]
}
```

```
06:28:13 /slack
{
  "channel": "#ml-quality",
  "username": "Alertmanager",
  "attachments": [
    {
      "title": "[RESOLVED] FeatureDriftKS \u2013 warning",
      "title_link": "http://68617e369671:9093/#/alerts?receiver=team4-ml-slack",
      "text": "\u2013KS-test drift on TotalCharges (D=0.000)*\nTwo-sample Kolmogorov\u2013Smirnov test rejects 'same distribution' (p<0.01, D>0.2).\nmodel: 'CustomerChurnGradientBoosting' feature: 'TotalCharges' since: 06:07:49\nrunbook: https://github.com/AutomationArtist01/MLOPS/blob/main/docs/Team4-Monitoring-Guidebook.pdf#featuredrift\n\nKS-test drift on tenure (D=0.000)*\nTwo-sample Kolmogorov\u2013Smirnov test rejects 'same distribution' (p<0.01, D>0.2).\nmodel: 'CustomerChurnGradientBoosting' feature: 'tenure' since: 06:07:19\nrunbook: https://github.com/AutomationArtist01/MLOPS/blob/main/docs/Team4-Monitoring-Guidebook.pdf#featuredrift\n",
      "fallback": "[RESOLVED] FeatureDriftKS CustomerChurnGradientBoosting warning (ml-quality drift-exporter ml-monitoring team4) | http://68617e369671:9093/#/alerts?receiver=team4-ml-slack",
      "callback_id": "",
      "footer": "",
      "color": "good",
      "mrkdwn_in": [
        "fallback",
        "pretext",
        "text"
      ]
    }
  ]
}
```

```
06:27:58 /slack
{
  "channel": "#ml-quality",
  "username": "Alertmanager",
  "attachments": [
    {
      "title": "[RESOLVED] FeatureDriftHigh \u2013 warning",
      "title_link": "http://68617e369671:9093/#/alerts?receiver=team4-ml-slack",
      "text": "\u2013DRIFT on MonthlyCharges: PSI 0.331 > 0.25*\nFeature distribution in the rolling window differs significantly from training data (model CustomerChurnGradientBoosting).\nmodel: 'CustomerChurnGradientBoosting' feature: 'MonthlyCharges' since: 06:06:34\nrunbook: https://github.com/AutomationArtist01/MLOPS/blob/main/docs/Team4-Monitoring-Guidebook.pdf#featuredrift\n\nDRIFT on TotalCharges: PSI 0.326 > 0.25*\nFeature distribution in the rolling window differs significantly from training data (model CustomerChurnGradientBoosting).\nmodel: 'CustomerChurnGradientBoosting' feature: 'TotalCharges' since: 06:07:19\nrunbook: https://github.com/AutomationArtist01/MLOPS/blob/main/docs/Team4-Monitoring-Guidebook.pdf#featuredrift\n\nDRIFT on tenure: PSI 0.319 > 0.25*\nFeature distribution in the rolling window differs significantly from training data (model CustomerChurnGradientBoosting).\nmodel: 'CustomerChurnGradientBoosting' feature: 'tenure' since: 06:06:34\nrunbook: https://github.com/AutomationArtist01/MLOPS/blob/main/docs/Team4-Monitoring-Guidebook.pdf#featuredrift\n",
      "fallback": "[RESOLVED] FeatureDriftHigh CustomerChurnGradientBoosting warning (ml-quality drift-exporter ml-monitoring team4) | http://68617e369671:9093/#/alerts?receiver=team4-ml-slack",
      "callback_id": "",
      "footer": "",
      "color": "good",
      "mrkdwn_in": [
        "fallback",
        "pretext",
        "text"
      ]
    }
  ]
}
```

Notifications — fake Slack (webhook catcher) and Mailhog inbox

## 5.3 A real finding

The other team's `api.py` fits its `ColumnTransformer` with transformers ordered *(categorical, numeric)* while the model was trained with *(numeric, categorical)*; because a NumPy array is passed to `predict`, scikit-learn cannot detect the mismatch. On the training CSV the correct order gives **AUC 0.94 / mean score 0.25**, the API's order **AUC 0.53 / mean score 0.09** — the live Render deployment silently returns scrambled predictions. Our stack surfaces exactly this class of bug through the prediction-distribution metrics (`PredictionDrift`, `PositiveRateAnomaly`) while every operational signal stays green; the fix in `services/model_api/api.py` is a one-line transformer reorder.

## 6 · Challenges

| Challenge                                                                                              | Resolution                                                                                     |
|--------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| Version pins of the other team's artifact (sklearn 1.9.0 + skops + MLflow 3.15)                        | matched exactly in <code>model_api</code> ; <code>uv pip compile</code> pins for every service |
| MLflow 3 rejects unknown <code>Host</code> headers ( <code>Invalid Host header</code> )                | <code>--allowed-hosts</code> for both the in-network name and the host port                    |
| macOS AirPlay occupies port 5000                                                                       | MLflow mapped to 5001                                                                          |
| <code>rate()</code> over a not-yet-existing 5xx series returns <i>empty</i> , not 0 → "No data" panels | zero-fill with <code>or (... * 0)</code> in recording rules                                    |
| Alertmanager has no env-var substitution                                                               | tiny entrypoint that <code>sed</code> s the config from env                                    |
| KS p-value → 0 with large n, alerts too eager                                                          | require effect size <code>D &gt; 0.2</code> as well                                            |
| Drift table in Grafana joins by labels                                                                 | aggregate by <code>(model, feature)</code> so <code>merge</code> aligns                        |
| A second monitored model with no traffic tripped <code>NoTraffic</code>                                | rule now means "traffic dropped" ( <code>max_over_time(... [30m]) &gt; 0.5</code> )            |
| Detection latency at low traffic (500-row window at 1.4 rps ≈ 6 min)                                   | documented; window size is a tuning knob ( <code>WINDOW_SIZE</code> )                          |
| Bug discovered in the wrapped model                                                                    | fixed in our copy, documented, reported to the other team                                      |

## 7 • Conclusion

---

The stack meets and exceeds the assignment: exporters for request rate, latency percentiles, error rate and prediction distribution; a custom PSI + KS drift exporter on a rolling window; Grafana dashboards and 14 alert rules routed to Slack and e-mail through Alertmanager; a one-command Docker Compose stack with the sample model API; and documentation (guidebook + wiring guide) that lets another team attach their model with environment variables only — demonstrated with a second, unmodified model. On top of the monitoring scope it ships a ZenML pipeline with a gating validation step, Optuna tuning tracked in MLflow with registry promotion, CI that lints, tests, validates configs, builds images and smoke-tests the whole stack, and a Render deployment of the model API.

**Next steps:** logs/traces (Loki/Tempo, OpenTelemetry), long-term storage (Thanos/Mimir), an accuracy exporter once ground-truth labels arrive, segmented drift, and an automatic retraining trigger from the `MultipleFeaturesDrifted` alert into the ZenML pipeline.

---

*Repository:* <https://github.com/AutomationArtist01/MLOPS> · *Guidebook:* [docs/Team4-Monitoring-Guidebook.pdf](#) · *Reproduce:* `make up && make train && make smoke`